Day 1 Student Notes - Welcome to Claude

Claude for DevOps Mastery. These notes cover everything from today's session, with examples you can reuse.

What we covered

Generative AI creates new content. Traditional software follows rules you write (if disk > 90%: send_alert()). Machine learning learns patterns from examples (a spam filter learns from millions of emails). Generative AI learns patterns and then generates new text, code or images ("write a script that rotates my log files").

An LLM predicts the next token. LLM means Large Language Model. It is trained on a huge amount of text and code, and its basic skill is: given the text so far, predict what comes next. If you type kubectl get, the most likely next word is pods. The model repeats this for every small piece of its answer.

How an answer is produced.

1. You send a prompt, for example "why is my pod crashing?"
2. The text is split into small chunks called tokens.
3. The model predicts the most likely next token.
4. The token is added to the text and the model predicts again, until the answer is complete.

Because it predicts plausible text, it can sound sure and still be wrong. It does not look facts up in a database.

Tokens and the context window. A token is a small chunk of text, roughly three-quarters of a word. Cost, speed and limits are counted in tokens (Day 2 goes deeper). The context window is everything Claude can see in one conversation: your prompts, attached files and its own replies. Think of it as working memory. Claude cannot see your servers unless you show it the output.

How a model like Claude is made. Step 1, pretraining: it learns language and code patterns from a very large amount of text. Step 2, training with feedback: it is taught to follow instructions and be helpful, honest and harmless, using human and AI feedback. Step 3, safety testing before release. A model also has a training cutoff, so very recent changes may be missing unless you provide the documentation.

Good at: writing and explaining scripts and configs, summarising logs and errors, converting formats (YAML to JSON), drafting docs, runbooks and commit messages.

Watch out for: hallucination (a confident answer that is wrong, such as an invented command flag), answers that vary from run to run, no view of your live systems, and a training cutoff.

> **Mental model:** Claude is a brilliant junior engineer who has read almost everything, and has never touched your servers. Fast and tireless. Needs context. Needs review.

Key words

Word	Meaning
Prompt	What you send to the model: instructions, context and files
LLM	Large Language Model. Predicts the next token from the text so far
Token	A small chunk of text, about three-quarters of a word. The unit of cost and limits
Context window	How much text the model can consider in one conversation
Hallucination	A confident answer that is wrong or invented
Training cutoff	The date after which the model has no training data
Verify	Check output against man pages, --help, official docs, or a test machine before using it

Anthropic, safety and the Claude model family

Claude is built by Anthropic, an AI safety and research company. Its stated goal is AI that is helpful, honest and harmless. Two methods to know by name: **Constitutional AI** (training with a written set of principles) and the **Responsible Scaling Policy** (stronger safeguards as models become more capable). Claude often flags risk and says when it is unsure, but it is not a safety net for production. You are.

Tier	Strength	Use it for
Haiku 5.5	Fastest, lowest cost	Log triage, quick command lookups
Sonnet 5.5	Balanced daily driver	Dockerfiles, manifests, Ansible, Terraform
Opus 5.5	Slower, deeper reasoning	Root-cause analysis, architecture review
Fable 5.1 (Mythos tier)	Most capable, premium	Specialist and select enterprise use

Rule: start with Sonnet. Go down for easy bulk work, go up for hard problems. Model names and prices change, so the instructor confirms the current line-up at the start of each batch.

Why DevOps teams use Claude

Four places where it saves real time:

- **Scripting speed:** a first draft of a Bash or Python script in seconds. Example: "Write a script that deletes log files older than 7 days from /var/log/myapp." You still read it before running it.
- **Infrastructure-as-code review:** a second pair of eyes on a Dockerfile, Terraform or Kubernetes file. Example: "Review this Dockerfile and list anything risky or wasteful."
- **Log and alert triage:** paste a short, sanitized error and get likely causes to check first. Example (illustrative): at 3 a.m. an alert says disk usage is 95%. Claude suggests

checking large folders with du, then old logs, then a growing temp directory. You run the checks and decide.
- **On-call and docs support:** explain an unfamiliar error, draft a runbook, write a clear commit message.

What it does not do: see your live systems, know your company's hidden rules, or approve changes to production. That part stays with you.

Prompts we tried

The four parts of a good prompt

Part	Example: planning a trip
Role	You are an experienced travel planner.
Context	I am going to Goa for 3 days in December. It is my first trip there.
Task	Make a packing list for me.
Format	10 short bullet points.

Weak prompt: "packing list". Claude has to guess where, when and for how long. In DevOps the same shape works: *You are a Linux admin. My Ubuntu server's root disk is 95% full. Give me 3 safe, read-only commands to find what is using the space, and explain each in one line.* We go deeper into prompting on Day 3.

Hands-on 1: weak prompt vs good prompt

Step 1: **What is cloud computing?** Step 2: in a new chat:

```
You are a friendly teacher. I am new to IT. Explain what cloud computing is using an everyday example, like renting a house. Answer in 5 short lines.
```

Compare the two answers. Which would you rather read?

Hands-on 2: improve the answer with follow-ups

```
Write a short, polite email to my manager asking for leave this Friday for a family function. Keep it under 80 words.
```

Then type, one by one: **Make it more formal.** / **Now make it 3 lines only.** / **Add that I will finish my work by Thursday.** You do not need the perfect prompt first time. Talk to Claude and refine it.

The habit: ask, read, verify, use safely

1. **Ask** with role, context, task and format.
2. **Read** the full answer. No blind copy and paste.
3. **Verify** key facts on an official website. For commands: man, --help or the official docs.
4. **Use safely.** Test machine first, never production first. No passwords or private data in chats.

Try it: ask Claude **Tell me 3 facts about my city, [your city].** Check one fact on Google or Wikipedia. Then ask: **Which of these facts should I double-check, and where?**

Optional, if you already use Linux

```
You are a senior Linux engineer helping a junior colleague. Explain this command
piece by piece and tell me what output to expect. Keep it under 150 words.

du -sh /var/log/* | sort -rh | head -5
```

What it does: lists the size of everything in /var/log, sorts largest first, and shows the top 5. It only reads, it changes nothing. Verify with **man du** or **du --help**.

Safety checklist before you paste anything into a chat

- No passwords, API keys, tokens or private keys.
- No customer data.
- No confidential company data or internal hostnames you are not allowed to share.
- When unsure, remove it or replace it with a placeholder like <API_KEY>.

Check yourself

Try these without looking, then compare with the answers.

1. What does an LLM do at its core?
2. Name the four parts of a good prompt.
3. Name the four Claude model tiers.
4. What is a hallucination?
5. Name one thing you must never paste into a chat.

Answers

1. It predicts the next token from the text so far. It does not look facts up.
2. Role, context, task, format.
3. Haiku, Sonnet, Opus, and the Mythos tier (currently Fable 5.1).
4. A confident answer that is wrong or invented.
5. Any of: passwords, API keys, tokens, customer data, confidential company data.

Take-home task (about 20 minutes)

1. Pick one everyday task, like a weekly study plan. Ask with a weak prompt, then a good one. Compare.
2. Ask Claude 3 facts you already know. Note anything wrong or missing.
3. List 2 tasks where AI saves you time and 2 where you would not trust it yet.
4. Read the plans and pricing page on claude.com. Do not subscribe yet.
5. Share in the batch group. No personal or company data.

Day 2 preview

Tokens, context windows and Claude pricing. Nothing to install. We will also cover the plans: Free, Pro, Max, Team and Enterprise.

Reminder: subscribe to Claude Pro on the evening of Day 3 so you are ready for Day 4. Check the current price and plan details on the official pricing page first, since they can change.